



**GENERALITAT
VALENCIANA**

Conselleria d'Educació,
Cultura i Esport



IES JUAN DE GARAY

C/ Juan de Garay, 25 46017 Valencia
Tel: 961 20 60 45 Fax: 961 20 60 46
46012963@gva.es www.juandegaray.es



GOBIERNO
DE ESPAÑA

MINISTERIO
DE EDUCACIÓN
Y FORMACIÓN PROFESIONAL



Unión Europea
Fondo Social Europeo
El FSE invierte en tu futuro

UD10

BD_UD10_BOLETÍN EJERCICIOS VARIOS_SOL

Bases de Datos
1º DAW/DAM

Abelardo Martínez
Elena González Moreno
Versión:260427.1034

Licencia



Reconocimiento - NoComercial - CompartirIgual (by-nc-sa): No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:

⚠ Importante

✿ Interesante

ÍNDICE DE CONTENIDO

1. Ejercicio 1: Trigger para actualizar stock.....	4
1.1 Para probar el Trigger.....	4
2. Trigger para registrar cambios de estado en pedidos.....	5
2.1 Para probar el Trigger.....	5
3. Trigger para evitar pedidos con cantidades negativas.....	6
3.1 Para probar el Trigger.....	6
4. Cursor para mostrar clientes de una ciudad.....	7
4.1 Cómo probar el cursor.....	8
5. Ejercicio 5: Cursor para calcular el total de compras de cada cliente.....	8
5.1 Cómo probar el cursor.....	9
6. Ejercicio 6: Cursor para listar productos con bajo stock.....	10
6.1 Cómo probar el cursor.....	11
7. Ejercicio 7: Cursor para contar empleados por oficina.....	12
7.1 Cómo probar el cursor.....	13
8. Ejercicio 8: Cursor para contar empleados por CIUDAD.....	14
8.1 Cómo probar el cursor.....	15
9. Ejercicio 9: Función para calcular el total de un pedido.....	15
9.1 Cómo probar la función.....	16

UD010. BD_UD10_BOLETÍN EJERCICIOS VARIOS_SOL

1. BASE DE DATOS A UTILIZAR

Cuidado se parece a UD05_jardineria pero no es igual.

La puedes encontrar en Recursos Complementarios del tema.

2. EJERCICIO 1: TRIGGER PARA ACTUALIZAR STOCK

Crear un trigger que, al insertar una nueva línea en la tabla detalle_pedido, reste la cantidad pedida del stock del producto correspondiente en la tabla productos.

```
DELIMITER //
```

```
CREATE TRIGGER tr_actualizar_stock
AFTER INSERT ON DetallePedidos
FOR EACH ROW
BEGIN
    UPDATE productos
    SET cantidadEnStock = cantidadEnStock - NEW.cantidad
    WHERE codigoProducto = NEW.codigoProducto;
END;
//
```

```
DELIMITER ;
```

2.1 Para probar el Trigger

-- Compruebas cuantos hay en stock

```
SELECT CantidadEnStock FROM Productos WHERE CodigoProducto='11679';
```

-- Insertas una nueva linea de detalles pedido con ese producto.

```
INSERT INTO DetallePedidos (CodigoPedido, CodigoProducto, Cantidad, PrecioUnidad,
NumeroLinea)
```

```
VALUES(1, '11679', 2, 7, 6);
```

```
-- Compruebas que hay 2 menos en stock
```

```
SELECT CantidadEnStock FROM Productos WHERE CodigoProducto='11679';
```

3. TRIGGER PARA REGISTRAR CAMBIOS DE ESTADO EN PEDIDOS

Crear un trigger que registre en una tabla llamada historial_estado_pedido cada vez que se actualice el estado (estado) de un pedido en la tabla pedidos. Esta tabla debe almacenar el codigoPedido, el estadoAnterior, el estadoNuevo, y la fechaCambioPie de foto.

```
CREATE TABLE historial_estado_pedido (
  id INT AUTO_INCREMENT PRIMARY KEY,
  codigoPedido INT,
  estadoAnterior VARCHAR(20),
  estadoNuevo VARCHAR(20),
  fechaCambio DATETIME
);
```

```
DELIMITER //
```

```
CREATE TRIGGER tr_estado_pedido
BEFORE UPDATE ON Pedidos
FOR EACH ROW
BEGIN
  IF OLD.estado <> NEW.estado THEN
    INSERT INTO historial_estado_pedido (codigoPedido, estadoAnterior, estadoNuevo,
fechaCambio)
      VALUES (OLD.codigoPedido, OLD.estado, NEW.estado, NOW());
  END IF;
END;
//
```

DELIMITER ;

3.1 Para probar el Trigger

//Asegúrate de tener un pedido en la tabla pedidos:

```
SELECT codigoPedido, estado FROM Pedidos LIMIT 1;
```

//Actualiza el estado de ese pedido:

```
UPDATE Pedidos  
SET estado = 'Devuelto'  
WHERE codigoPedido = 1;
```

//Comprueba que se haya registrado el cambio:

```
SELECT * FROM historial_estado_pedido WHERE codigoPedido = 1;
```

4. TRIGGER PARA EVITAR PEDIDOS CON CANTIDADES NEGATIVAS

Crear un trigger que impida insertar líneas en la tabla detalle_pedido si la cantidad de productos es menor o igual a cero. Si se intenta, debe lanzar un error.

DELIMITER //

```
CREATE TRIGGER tr_validar_cantidad_pedido  
BEFORE INSERT ON DetallePedidos  
FOR EACH ROW  
BEGIN  
  IF NEW.cantidad <= 0 THEN  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT = 'La cantidad del producto debe ser mayor que cero';  
  END IF;
```

```
END;  
//  
DELIMITER ;
```

4.1 Para probar el Trigger

-- Intenta insertar una línea con cantidad negativa:

```
INSERT INTO DetallePedidos (codigoPedido, codigoProducto, cantidad, precioUnidad,  
numeroLinea)
```

```
VALUES (1, 'FR-123', -5, 10.5, 1);
```

-- Verás el siguiente error:

ERROR 1644 (45000): La cantidad del producto debe ser mayor que cero.

--Intenta con un valor válido para comprobar que sí se permite:

```
INSERT INTO DetallePedidos (codigoPedido, codigoProducto, cantidad, precioUnidad,  
numeroLinea)
```

```
VALUES (1, 'FR-105', 5, 10.5, 2);
```

5. CURSOR PARA MOSTRAR CLIENTES DE UNA CIUDAD

Crear un procedimiento almacenado que reciba como parámetro el nombre de una ciudad y recorra, usando un cursor, todos los clientes de esa ciudad mostrando su nombre y su teléfono mediante SELECT.

```
DELIMITER //
```

```
CREATE PROCEDURE mostrar_clientes_ciudad(IN ciudad_busqueda VARCHAR(50))
```

```
BEGIN
```

```
    DECLARE terminado INT DEFAULT 0;
```

```
    DECLARE nombre_cliente VARCHAR(50);
```

```
    DECLARE telefono_cliente VARCHAR(20);
```

```
    DECLARE cur CURSOR FOR
```

```
        SELECT nombreCliente, telefono
```

```
        FROM Clientes
```

```
        WHERE ciudad = ciudad_busqueda;
```

```
DECLARE CONTINUE HANDLER FOR NOT FOUND SET terminado = 1;
```

```
OPEN cur;
```

```
leer_clientes: LOOP
```

```
  FETCH cur INTO nombre_cliente, telefono_cliente;
```

```
  IF terminado = 1 THEN
```

```
    LEAVE leer_clientes;
```

```
  END IF;
```

```
  SELECT nombre_cliente AS Nombre, telefono_cliente AS Telefono;
```

```
END LOOP;
```

```
CLOSE cur;
```

```
END;
```

```
//
```

```
DELIMITER ;
```

5.1 Cómo probar el cursor

-- llamar al procedimiento con una ciudad real (por ejemplo "Madrid"):

```
CALL mostrar_clientes_ciudad('Madrid');
```

El resultado serán varias consultas SELECT que devuelven los nombres y teléfonos de los clientes que viven en esa ciudad.

6. EJERCICIO 5: CURSOR PARA CALCULAR EL TOTAL DE COMPRAS DE CADA CLIENTE.

Crear un procedimiento almacenado que recorra todos los clientes y calcule el total gastado por cada uno en todos sus pedidos. Mostrar el nombre del cliente junto con el total gastado.

Pista: Debes usar las tablas *Clientes*, *Pedidos* y *DetallePedidos*.

Se usa IFNULL para devolver 0 si el cliente no ha hecho pedidos aún.

La acumulación se hace con una consulta individual dentro del bucle.

DELIMITER //

CREATE PROCEDURE total_por_cliente()

BEGIN

DECLARE terminado INT DEFAULT 0;

DECLARE id_cliente INT;

DECLARE nombre_cliente VARCHAR(50);

DECLARE total_cliente DECIMAL(10,2);

DECLARE cur CURSOR FOR

SELECT codigoCliente, nombreCliente FROM Clientes;

DECLARE CONTINUE HANDLER FOR NOT FOUND SET terminado = 1;

OPEN cur;

recorrer_clientes: LOOP

FETCH cur INTO id_cliente, nombre_cliente;

IF terminado = 1 THEN

LEAVE recorrer_clientes;

END IF;

*SELECT SUM(dp.cantidad * dp.precioUnidad) INTO total_cliente*

FROM Pedidos p

JOIN DetallePedidos dp ON p.codigoPedido = dp.codigoPedido

WHERE p.codigoCliente = id_cliente;

SELECT nombre_cliente AS Cliente, IFNULL(total_cliente, 0) AS Total_Gastado;

END LOOP;

CLOSE cur;

END;

```
//
```

```
DELIMITER ;
```

6.1 Cómo probar el cursor

```
-- Llamada
```

```
CALL total_por_cliente();
```

Verás un conjunto de resultados con el nombre de cada cliente y el total gastado en todos sus pedidos.

7. EJERCICIO 6: CURSOR PARA LISTAR PRODUCTOS CON BAJO STOCK

Crear un procedimiento almacenado que recorra todos los productos usando un cursor y muestre aquellos cuya cantidad en stock sea inferior a 50 unidades, mostrando su nombre, código y cantidad disponible.

```
DELIMITER //
```

```
CREATE PROCEDURE productos_bajo_stock()
```

```
BEGIN
```

```
    DECLARE terminado INT DEFAULT 0;
```

```
    DECLARE codigo VARCHAR(15);
```

```
    DECLARE nombre VARCHAR(70);
```

```
    DECLARE stock INT;
```

```
    DECLARE cur CURSOR FOR
```

```
        SELECT codigoProducto, nombre, cantidadEnStock FROM Productos;
```

```
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET terminado = 1;
```

```
    OPEN cur;
```

```
    recorrer_productos: LOOP
```

```
        FETCH cur INTO codigo, nombre, stock;
```

```
        IF terminado = 1 THEN
```

```
    LEAVE recorrer_productos;  
END IF;
```

```
IF stock < 50 THEN  
    SELECT codigo AS Codigo, nombre AS NombreProducto, stock AS StockDisponible;  
END IF;  
END LOOP;
```

```
CLOSE cur;  
END;  
//
```

```
DELIMITER ;
```

7.1 Cómo probar el cursor

-- Llama al procedimiento:

```
CALL productos_bajo_stock();
```

Obtendrás una serie de resultados con los productos que tienen bajo nivel de stock.

8. EJERCICIO 7: CURSOR PARA CONTAR EMPLEADOS POR OFICINA.

Crear un procedimiento almacenado que recorra todas las oficinas usando un cursor y muestre el nombre de la ciudad y la cantidad de empleados que trabajan en cada una de las oficinas, indicando la ciudad.

```
DELIMITER //
```

```
CREATE PROCEDURE contar_empleados_por_oficina()
```

```
BEGIN
```

```
    DECLARE terminado INT DEFAULT 0;
```

```
    DECLARE v_cod_oficina VARCHAR(10);
```

```
    DECLARE v_ciudad VARCHAR(50);
```

```
    DECLARE v_total_empleados INT;
```

```
    DECLARE cur CURSOR FOR
```

```
        SELECT codigoOficina, ciudad FROM Oficinas;
```

```
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET terminado = 1;
```

```
    OPEN cur;
```

```
    recorrer_oficinas: LOOP
```

```
        FETCH cur INTO v_cod_oficina, v_ciudad;
```

```
        IF terminado = 1 THEN
```

```
            LEAVE recorrer_oficinas;
```

```
        END IF;
```

```
        SELECT COUNT(*) INTO v_total_empleados
```

```
        FROM Empleados
```

```
        WHERE codigoOficina = v_cod_oficina;
```

```
        SELECT CONCAT("En la oficina: ", v_cod_oficina , " de la ciudad: " ,v_ciudad, " hay ",  
v_total_empleados, " empleados");
```

```
    END LOOP;
```

```
    CLOSE cur;
```

```
END;
```

```
//
```

DELIMITER ;

8.1 Cómo probar el cursor

CALL contar_empleados_por_oficina();

-- Verás un conjunto de resultados con oficinas, ciudades y número de empleados por oficina indicando de qué ciudad son.

-- Si quieres hacer una prueba manual:

SELECT * FROM oficinas;

SELECT * FROM empleados WHERE codigoOficina = 'MAD-ES';

9. EJERCICIO 8: CURSOR PARA CONTAR EMPLEADOS POR CIUDAD.

Crear un procedimiento almacenado que recorra todas las oficinas usando un cursor y muestre el nombre de la ciudad y la cantidad de empleados que trabajan en cada una DE LAS CIUDADES.

```
DELIMITER //
CREATE PROCEDURE emplePorCiudad ()
BEGIN
    DECLARE terminado boolean default false;
    DECLARE v_ciudad VARCHAR(30);
    DECLARE v_numEmpleados int;

    DECLARE cur cursor for
    SELECT distinct ciudad FROM Oficinas;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET terminado = true;

    OPEN cur;
    REPEAT
        FETCH cur INTO v_ciudad;

        IF not terminado then
            SELECT Count(*) into v_numEmpleados
            FROM Empleados e, Oficinas o
            WHERE e.CodigoOficina = o.CodigoOficina
            and o.ciudad = v_ciudad;

            SELECT CONCAT("En la ciudad: ",v_ciudad, " hay ", v_numEmpleados, " empleados");
        end if;

    UNTIL terminado END REPEAT;
    CLOSE cur;
```

```
END;  
// DELIMITER ;  
-- borrar  
drop procedure emplePorCiudad;  
-- Llamada  
Call emplePorCiudad();
```

9.1 Cómo probar el cursor

```
CALL emplePorCiudad();
```

-- Verás un conjunto de resultados con ciudades y número de empleados por ciudad.

10. EJERCICIO 9: FUNCIÓN PARA CALCULAR EL TOTAL DE UN PEDIDO.

Crear una función que reciba el `codigoPedido` como parámetro y devuelva el total del importe de ese pedido ($\text{cantidad} \times \text{precioUnidad}$ de cada línea).

```
DELIMITER //  
  
CREATE FUNCTION total_pedido(cod_pedido INT)  
RETURNS DECIMAL(10,2)  
NO DETERMINISTIC  
READS SQL DATA  
BEGIN  
    DECLARE total DECIMAL(10,2);  
  
    SELECT SUM(cantidad * precioUnidad)  
    INTO total  
    FROM DetallePedidos  
    WHERE codigoPedido = cod_pedido;  
  
    RETURN IFNULL(total, 0);  
END;  
//
```

DELIMITER ;

10.1 Cómo probar la función

-- Llama a la función con el ID de un pedido existente:

```
SELECT total_pedido(1);
```

-- Para ver qué contiene ese pedido:

```
SELECT * FROM detalle_pedido WHERE codigoPedido = 1;
```